Báo cáo bài tập E1-E3

Học sâu và ứng dụng trong thị giác máy tính - CO5085

Học kỳ 261 (2026-2027)

Giảng viên: Lê Thành Sách, Nguyễn Quốc Minh

Học viên	Mã số học viên
Nguyễn Xuân Trường	2570526
Trần Lê Hoàng Long	2570443

Bản này gồm E1 và E2. E3 sẽ được bổ sung vào cùng file ở hạn nộp tiếp theo.

Repository: SDH_HK261_CO5085.

Mục lục

1. Dữ liệu và cách đánh giá **3**

1.1. Fashion-MNIST . . . 3

1.2. Split và tiền xử lý . . . 3

1.3. Cách đánh giá và môi trường . . . 5

2. E1 - Softmax, MLP và CNN **7**

2.1. Kiến trúc và cấu hình . . . 7

2.2. Kết quả . . . 7

2.3. Phân tích lỗi . . . 8

3. E2 - Transformer: MSA và cách lấy token **11**

3.1. Mục tiêu và phạm vi . . . 11

3.2. Kiến trúc và công thức . . . 11

3.3. Bốn cách lấy token . . . 13

3.4. Thiết kế thí nghiệm và huấn luyện . . . 14

3.5. Kiểm chứng MSA tự viết . . . 14

3.6. Kết quả so sánh . . . 15

3.7. Phân tích lỗi và hình attention . . . 17

3.8. Kết luận E2 . . . 18

4. So sánh E1 và E2 **19**

Phụ lục **20**

P.1. Nguồn số liệu . . . 20

P.2. Khai báo hỗ trợ AI . . . 20

Tài liệu tham khảo **21**

1. Dữ liệu và cách đánh giá

1.1. Fashion-MNIST

E1 và E2 dùng chung Fashion-MNIST: ảnh xám 28×28, mười lớp, gồm 60.000 ảnh train và 10.000 ảnh test [1]. Nhóm chọn tập này vì nhỏ, chạy nhanh khi so nhiều mô hình, và có các lớp áo khá giống nhau nên khó hơn MNIST.

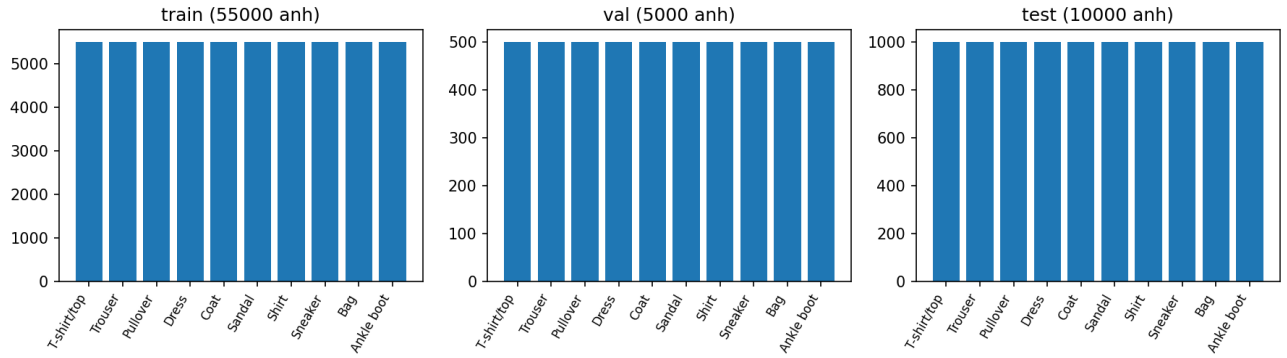
ID	Nhãn	Diễn giải
0	T-shirt/top	Áo thun
1	Trouser	Quần dài
2	Pullover	Áo chui đầu
3	Dress	Váy liền
4	Coat	Áo khoác
5	Sandal	Dép xăng đan
6	Shirt	Áo sơ mi
7	Sneaker	Giày thể thao
8	Bag	Túi
9	Ankle boot	Bốt cổ thấp

1.2. Split và tiền xử lý

Code E1 lấy 500 ảnh mỗi lớp từ tập train làm validation (seed 42) và lưu index vào E1/splits/. E2 đọc lại đúng các file index này.

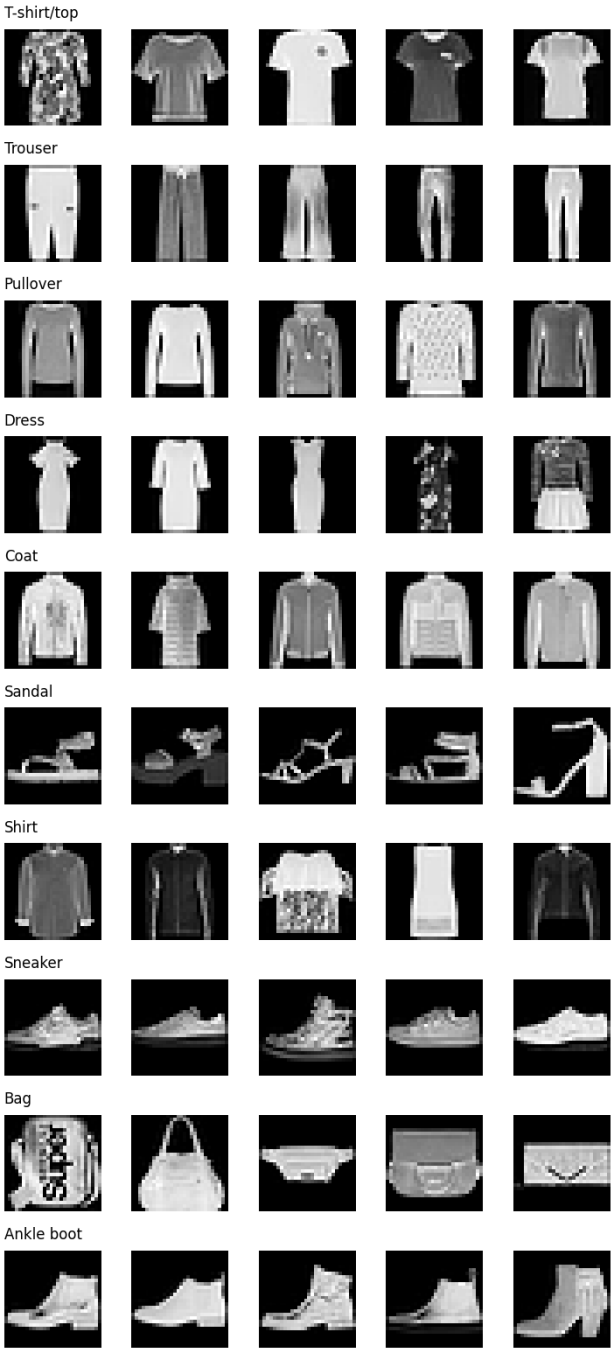
Split	Tổng ảnh	Ảnh/lớp	Vai trò
Train	55.000	5.500	Cập nhật trọng số
Validation	5.000	500	Chọn checkpoint
Test	10.000	1.000	Báo cáo kết quả cuối

Train dùng RandomCrop(28, padding=2), ToTensor và Normalize(mean=0,2860, std=0,3530). Validation và test bỏ RandomCrop.



Hình 1.1. Số ảnh mỗi lớp theo ba split.

Các lớp có số ảnh bằng nhau, nên accuracy không bị lệch về một lớp đông ảnh.



Hình 1.2. Năm ảnh train đầu tiên của mỗi lớp.

Ở độ phân giải này, các lớp áo có đường viền khá giống nhau. Các mục 2.3 và 3.7 đối chiếu nhận xét này với confusion matrix.

1.3. Cách đánh giá và môi trường

Mỗi mô hình lấy checkpoint có validation accuracy cao nhất, rồi đánh giá một lần trên test. Macro-F1 là trung bình F1 của mười lớp. Chênh lệch giữa hai accuracy được ghi bằng điểm phần trăm. Mỗi cấu hình chỉ chạy một lần với seed 42, nên các chênh lệch nhỏ chưa đủ để kết luận.

Mục	E1	E2
Nền tảng	Google Colab	Google Colab
GPU	NVIDIA Tesla T4	NVIDIA A100-SXM4-40GB
PyTorch	2.11.0+cu130	2.11.0+cu130

Hai bài chạy trên GPU khác nhau, nên thời gian của E1 và E2 không so trực tiếp được.

2. E1 - Softmax, MLP và CNN

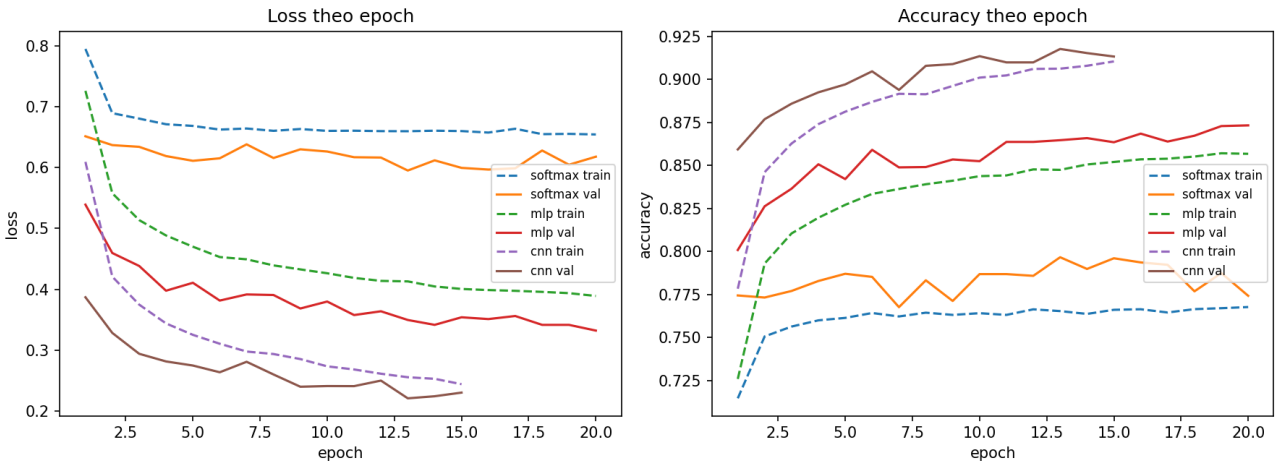
2.1. Kiến trúc và cấu hình

Mô hình	Kiến trúc	Tham số	Epoch
Softmax	Linear 784 → 10	7.850	20
MLP	Linear 784 → 256 → 128 → 10, ReLU, dropout 0,2	235.146	20
CNN	Hai khối Conv-BatchNorm-ReLU-MaxPool (32, 64 kênh), Linear 3.136 → 128 → 10, dropout 0,3	421.834	15

Cả ba mô hình dùng Adam với learning rate 0,001, CrossEntropyLoss, batch 128, seed 42 và training loop tự viết. Checkpoint được chọn theo validation accuracy cao nhất.

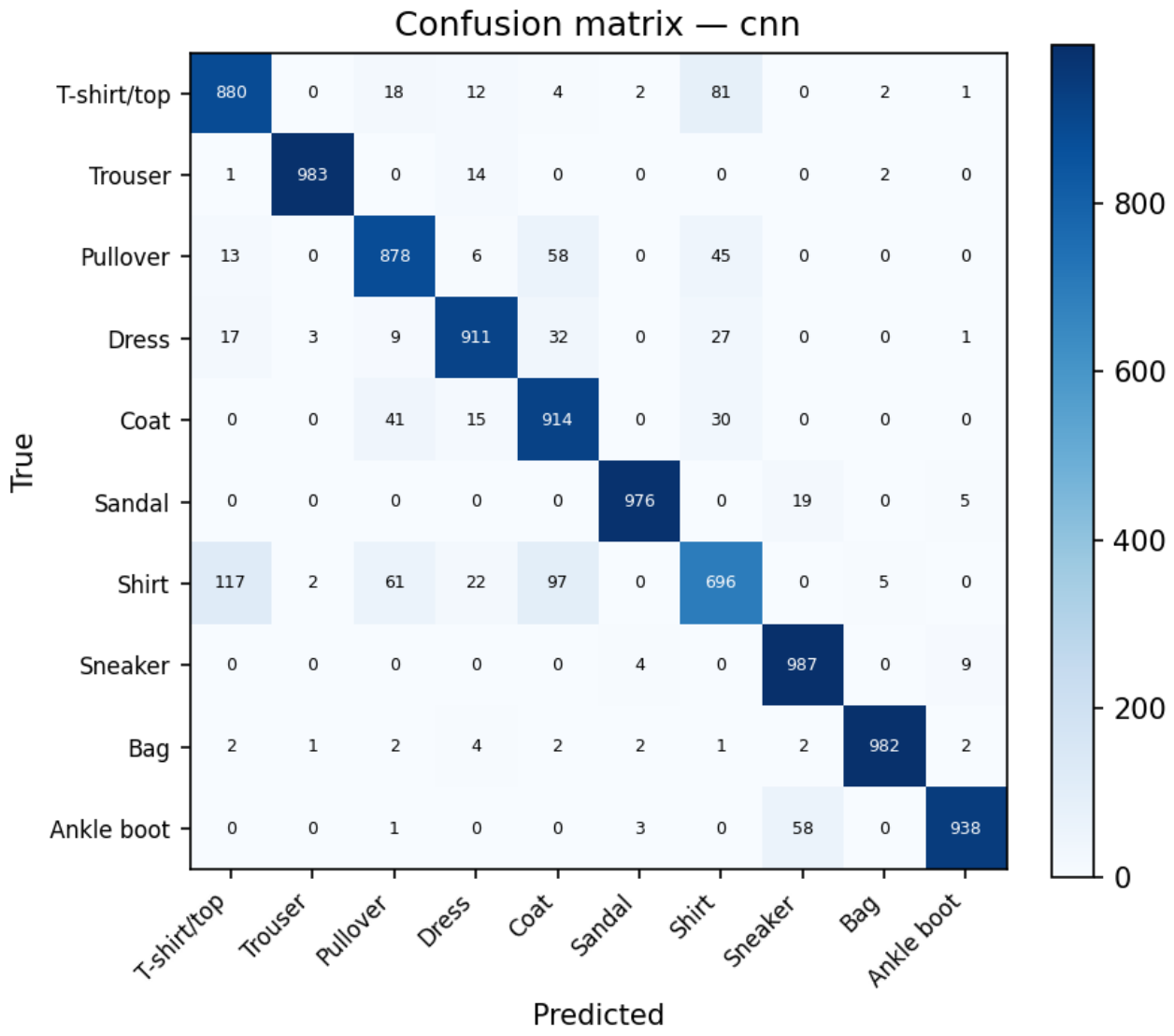
2.2. Kết quả

Mô hình	Epoch tốt nhất	Val (%)	Test (%)	Macro-F1 (%)	Thời gian (s)
Softmax	13	79,66	79,05	78,78	295,05
MLP	20	87,32	87,39	87,36	299,68
CNN	13	91,76	91,45	91,39	236,18



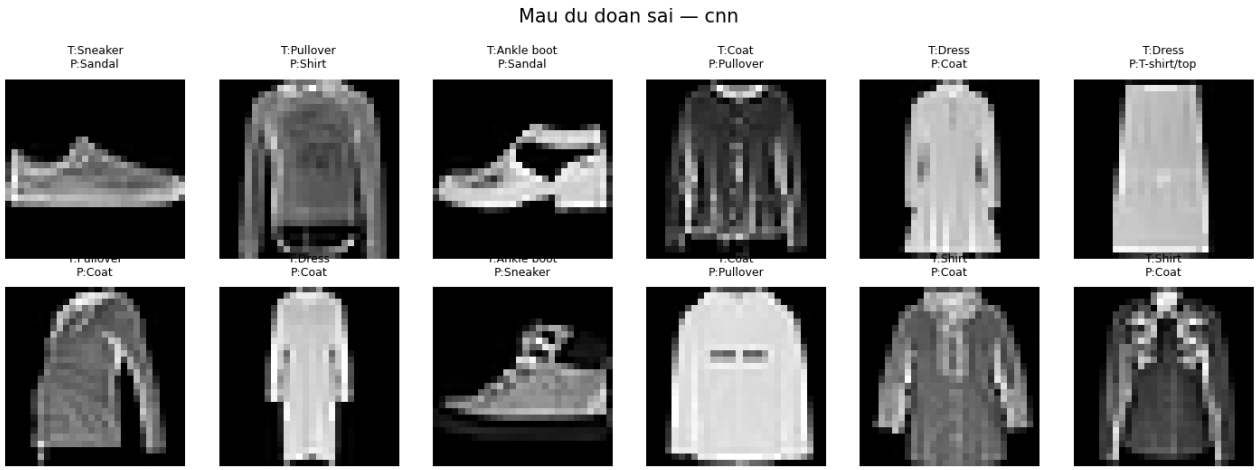
Hình 2.1. Loss và accuracy theo epoch của ba mô hình.

2.3. Phân tích lỗi

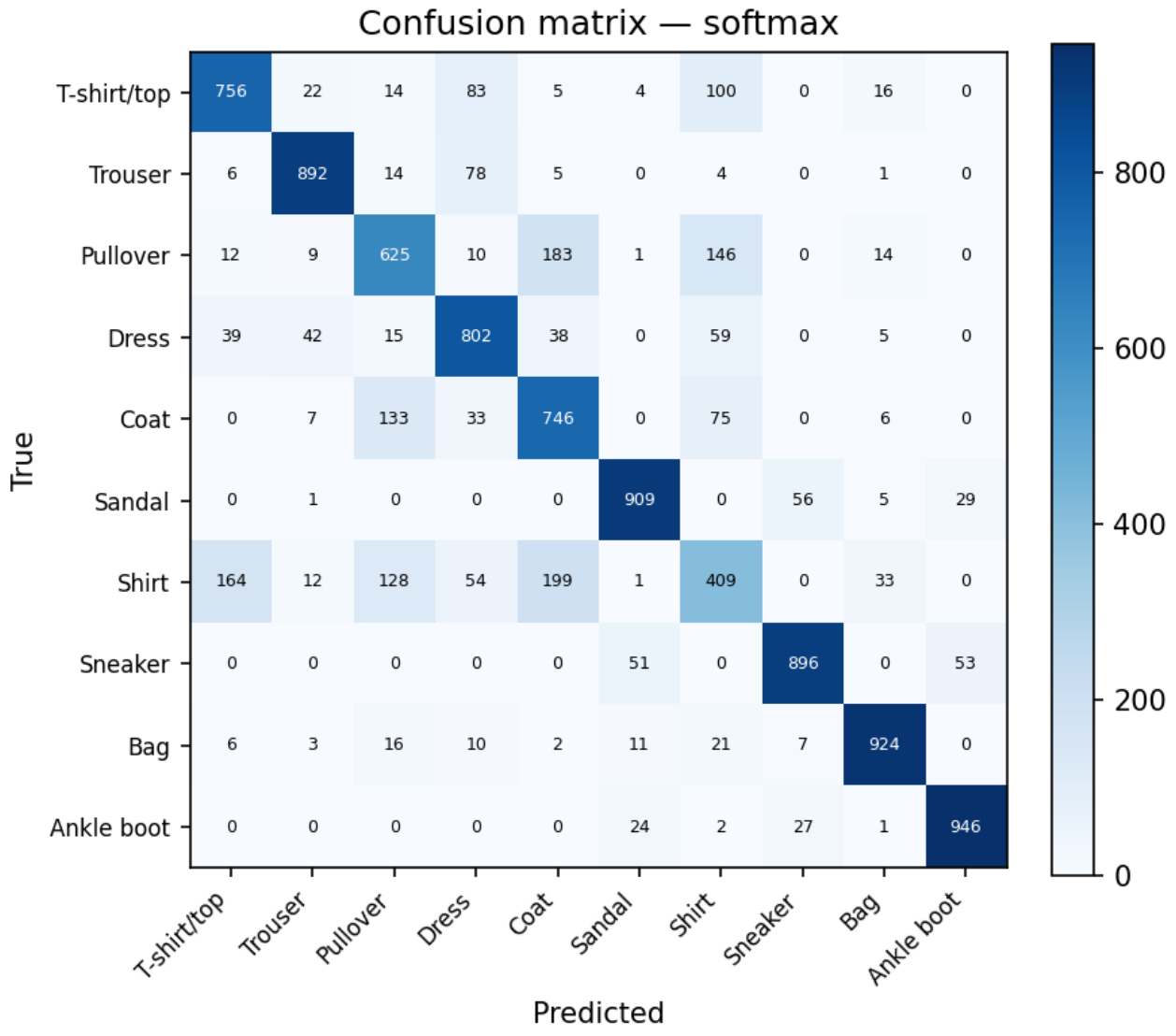


Hình 2.2. Confusion matrix của CNN.

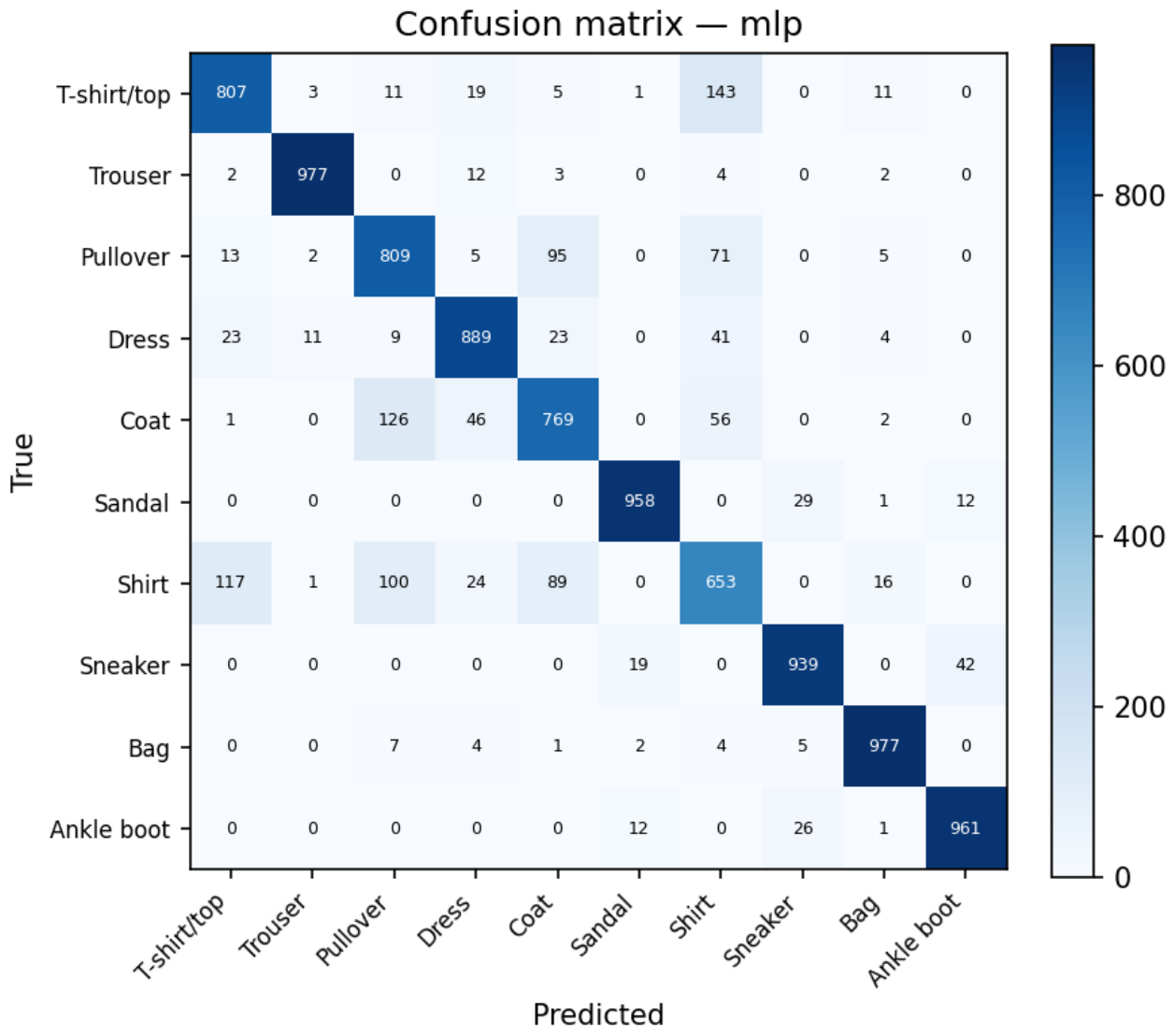
CNN đúng 9.145 và sai 855 ảnh trên 10.000. Các ô nhầm lớn nhất là Shirt → T-shirt/top (117 ảnh), Shirt → Coat (97 ảnh) và T-shirt/top → Shirt (81 ảnh). Shirt có recall thấp nhất, 69,60%.



Hình 2.3. Một số ảnh CNN dự đoán sai, T là nhãn thật và P là dự đoán.



Hình 2.4. Confusion matrix của Softmax.



Hình 2.5. Confusion matrix của MLP.

3. E2 - Transformer: MSA và cách lấy token

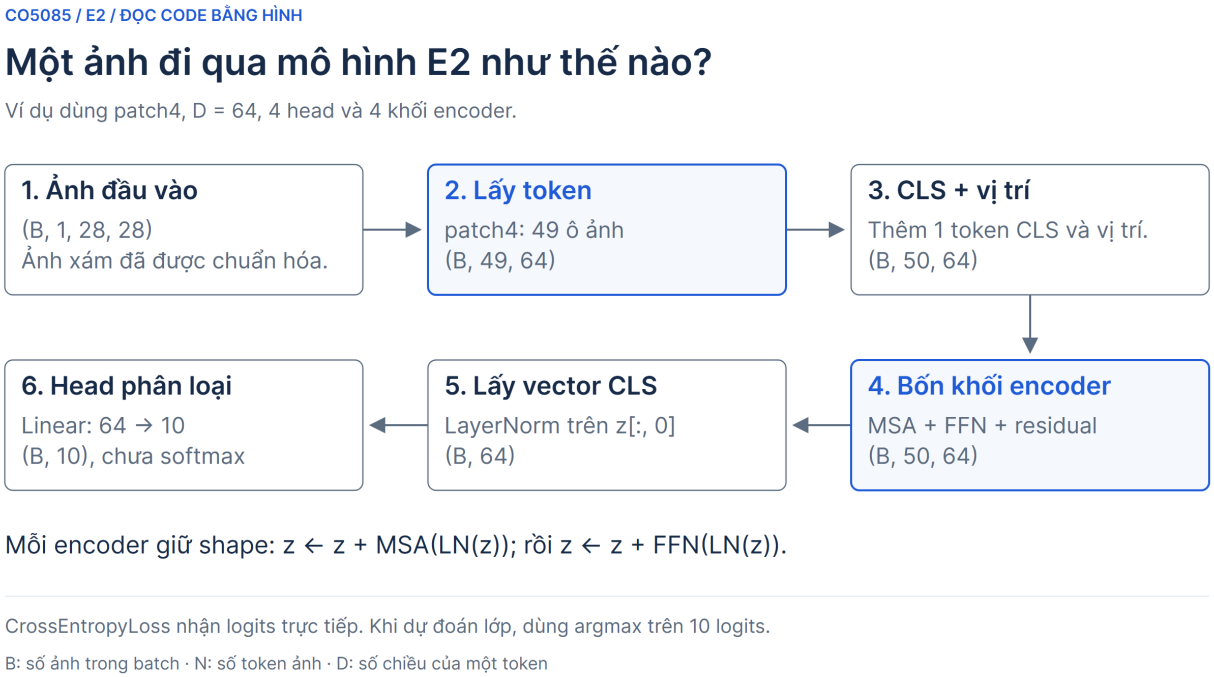
3.1. Mục tiêu và phạm vi

Bài E2 phân loại Fashion-MNIST bằng một bộ mã hóa Transformer nhỏ, không dùng mô hình pretrained. Báo cáo trả lời hai câu hỏi: (i) MSA tự viết và MSA của PyTorch khác nhau thế nào khi cùng dùng patch 4×4; (ii) cách biến ảnh thành chuỗi token ảnh hưởng ra sao khi cùng dùng MSA tự viết. Đề yêu cầu ít nhất hai cách lấy token; nhóm cài đặt bốn cách, tổng cộng năm thí nghiệm.

Trong MSA, mỗi token tổng hợp thông tin từ các token khác theo trọng số tính từ chính đầu vào. Ở bài này, token là vector mô tả một ô ảnh, một hàng ảnh hoặc một vị trí trên feature map. Cách dùng chuỗi patch lấy ý tưởng từ ViT [3], với mô hình nhỏ hơn nhiều.

3.2. Kiến trúc và công thức

Ảnh đầu vào có dạng (B,1,28,28), với B là số ảnh trong batch. Tokenizer tạo (B,N,64). Mô hình thêm một vector CLS học được vào đầu chuỗi và cộng vector vị trí học được, nên độ dài đi vào attention là S=N+1.



Hình 3.1. Luồng dữ liệu của bộ phân loại Transformer.

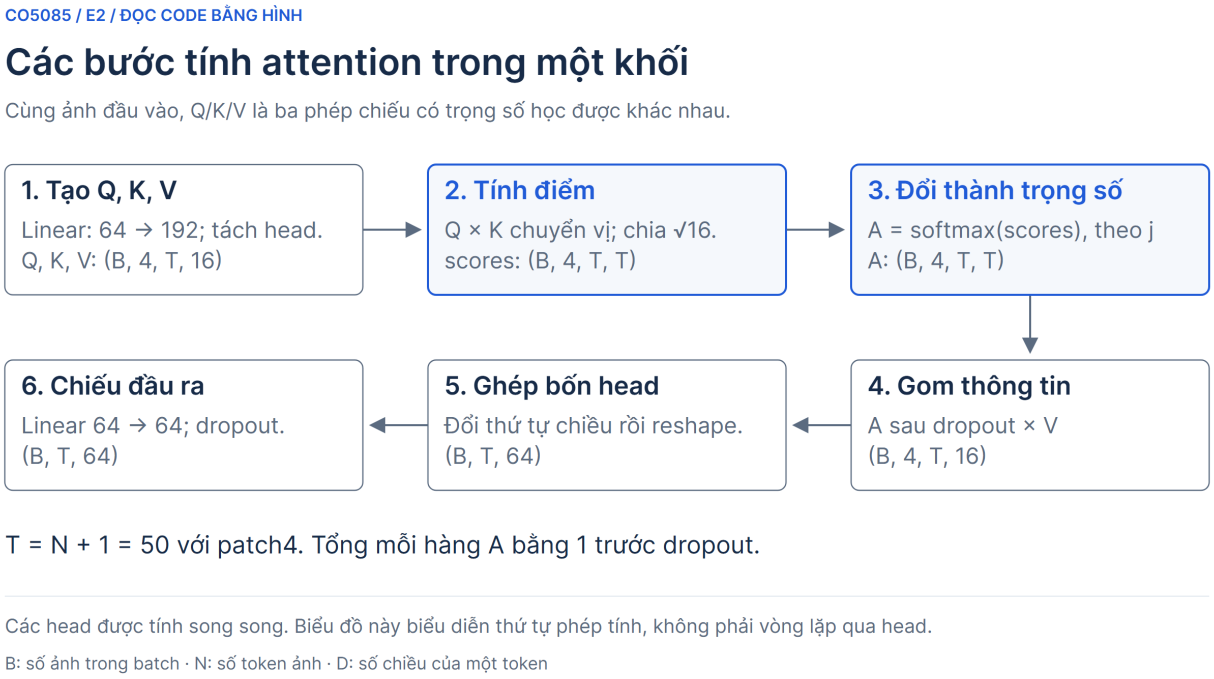
Mô hình có bốn khối encoder dạng pre-norm:

```
u = z + MSA(LayerNorm(z))
z_next = u + FFN(LayerNorm(u))
FFN(x) = Linear_2(Dropout(GELU(Linear_1(x)))) rồi Dropout
```

FFN đổi chiều 64 → 128 → 64 trên từng token, còn MSA trao đổi thông tin giữa các token. Cuối encoder, token CLS đi qua LayerNorm và Linear 64 → 10 để tạo logits. CrossEntropyLoss nhận logits trực tiếp.

Trong mỗi head, với chiều head d=64/4=16 [2]:

```
Q = X W_Q + b_Q; K = X W_K + b_K; V = X W_V + b_V
A = softmax(Q K^T / sqrt(d)), softmax theo chiều key
head = A V
MSA(X) = concat(head_1, head_2, head_3, head_4) W_O + b_O
```

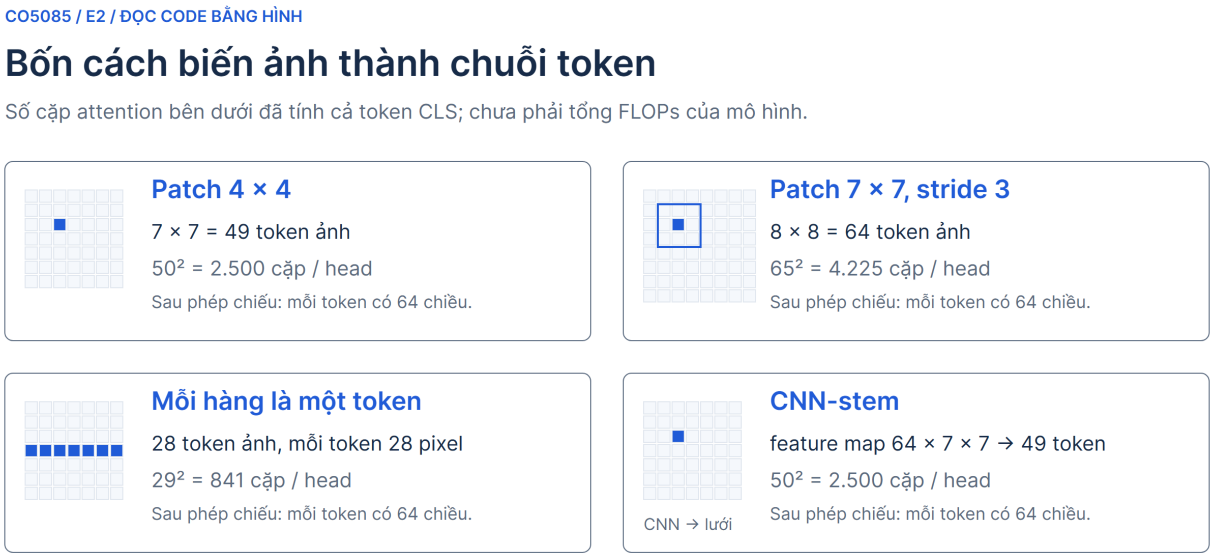


Hình 3.2. Các bước tính attention và kích thước tensor.

ScratchMSA gộp ba phép chiếu thành một Linear 64 → 192, tách Q/K/V và bốn head, rồi dùng einsum để tính điểm và tổng có trọng số. Bản này không gọi nn.MultiheadAttention hay nn.TransformerEncoder. TorchMSA gọi nn.MultiheadAttention với batch_first=True và truyền cùng x cho query, key, value [4].

Đại lượng với patch4	Kích thước
Ảnh	B × 1 × 28 × 28
Patch đã trải phẳng	B × 49 × 16
Token sau Linear	B × 49 × 64
Chuỗi sau thêm CLS	B × 50 × 64
Mỗi Q, K, V sau tách head	B × 4 × 50 × 16
Điểm / trọng số attention	B × 4 × 50 × 50
Kết quả ghép head	B × 50 × 64
Logits phân loại	B × 10

3.3. Bốn cách lấy token



Hình 3.3. Bốn cách biểu diễn ảnh thành token.

Cách	Phần ảnh trước phép chiếu	N	S=N+1	S² mỗi head
patch4	Ô 4×4, stride 4	49	50	2.500
patch7s3	Ô 7×7, stride 3	64	65	4.225
row	Một hàng gồm 28 pixel	28	29	841
cnnstem	Một vị trí trong feature map 7×7	49	50	2.500

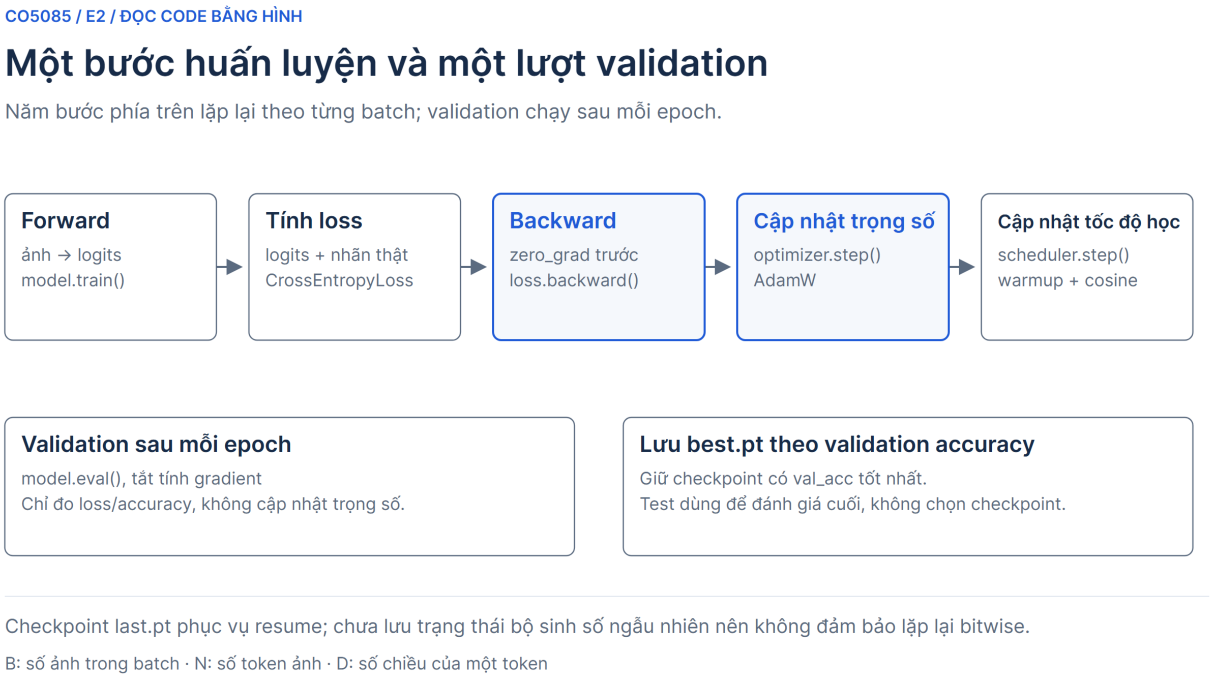
Với patch, số ô theo một chiều là floor((28-p)/s)+1, không padding. Patch4 cho lưới 7×7. Patch7s3 cho lưới 8×8 với các ô chồng lấn. Row chiếu mỗi hàng 28 → 64, ít cặp attention nhất nhưng nén toàn bộ thông tin ngang của hàng vào một vector.

CNN-stem gồm Conv 1 → 32, BatchNorm, ReLU, MaxPool rồi Conv 32 → 64, BatchNorm, ReLU, MaxPool. Feature map 64×7×7 được trải thành 49 token. Vì có thêm các lớp convolution, thí nghiệm này thay đổi cả số tham số và loại phép biến đổi, ngoài cách chia token.

3.4. Thiết kế thí nghiệm và huấn luyện

Thuộc tính dùng chung	Giá trị
Chiều token / số khối / số head	64 / 4 / 4
FFN / dropout	64 → 128 → 64 / 0,1
Optimizer	AdamW
Learning rate cơ sở / weight decay	0,001 / 0,05
Epoch / batch / seed	30 / 128 / 42
Lịch learning rate	Warmup 2 epoch rồi cosine theo từng batch
Dữ liệu	Train 55.000, validation 5.000, test 10.000
Chọn checkpoint	Validation accuracy cao nhất

Training loop tự viết theo thứ tự forward → loss → zero_grad → backward → optimizer.step → scheduler.step. Khi validation, mô hình chuyển sang eval và tắt gradient. Loss được cộng theo số ảnh mỗi batch rồi chia tổng số ảnh, vì batch cuối có ít ảnh hơn.



Hình 3.4. Một bước học và cách chọn checkpoint.

best.pt chứa trọng số, epoch và validation accuracy. last.pt có thêm optimizer và scheduler để train tiếp khi Colab bị ngắt. Cả năm thí nghiệm đều chạy đủ 30 epoch trên toàn bộ tập train.

3.5. Kiểm chứng MSA tự viết

Nhóm chép trọng số Q/K/V và đầu ra từ TorchMSA sang ScratchMSA, chuyển cả hai sang eval, cho cùng đầu vào rồi so output, attention từng head và gradient theo đầu vào.

Ca	B, S, D, số head	Max sai lệch output	Attention	Gradient đầu vào	Kết quả
1	8, 50, 64, 4	0	0	0	Đạt
2	8, 29, 32, 8	0	0	0	Đạt

Ngưỡng đạt là 1e-5 cho output và attention, 1e-4 cho gradient; tổng mỗi hàng attention cũng phải bằng 1. Phép kiểm tra chạy trên CPU, ở chế độ eval, không dùng dropout hay mask. tests/test_msa.py so thêm gradient của các tham số QKV và out_proj.

3.6. Kết quả so sánh

Từng mô hình được đánh giá bằng checkpoint có validation accuracy cao nhất.

Thí nghiệm	Tham số	Val tốt nhất (%)	Test (%)	Macro-F1 (%)	Epoch tốt nhất
patch4_scratch	139.018	88,00	88,50	88,42	28
patch4_torch	139.018	88,32	88,54	88,40	28
patch7s3_scratch	142.090	88,82	89,54	89,48	27
row_scratch	138.442	89,42	89,14	89,05	30
cnnstem_scratch	156.938	92,46	92,25	92,22	30

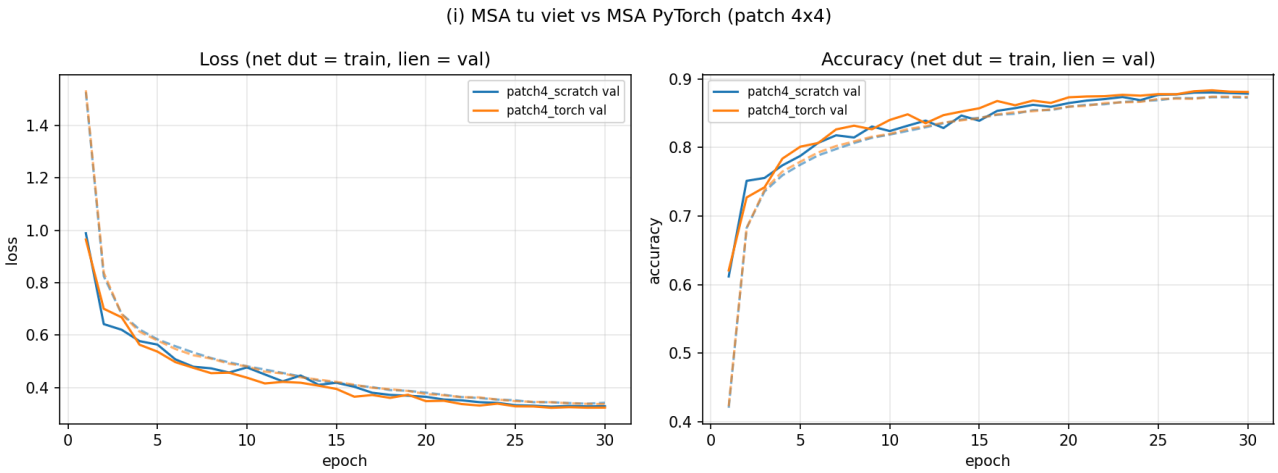
Thí nghiệm	Train tổng (s)	Train/epoch (s)	Suy luận gồm nạp dữ liệu (ảnh/s)
patch4_scratch	258,17	8,61	8.909,01
patch4_torch	267,12	8,90	8.801,84
patch7s3_scratch	264,58	8,82	8.596,72
row_scratch	255,79	8,53	8.906,62
cnnstem_scratch	260,00	8,67	8.657,80

Thời gian train chỉ tính vòng train, không gồm validation và ghi checkpoint. Tốc độ suy luận còn gồm DataLoader và chuyển dữ liệu sang GPU.

(i) MSA tự viết và PyTorch với cùng patch4

Hai mô hình có cùng số tham số và kết quả gần như nhau: test 88,50% và 88,54%, chênh 4 ảnh trên 10.000. Macro-F1 của bản tự viết nhỉnh hơn 0,02 điểm, validation của bản PyTorch cao hơn 0,32 điểm. Với một seed, các chênh lệch này chưa cho thấy bản nào tốt hơn. Hai bản cũng khởi tạo khác nhau: nn.MultiheadAttention dùng Xavier, còn bản tự viết dùng mặc định của nn.Linear.

Thời gian mỗi epoch là 8,61 giây với bản tự viết và 8,90 giây với bản PyTorch, chênh khoảng 3%. Epoch đầu của bản PyTorch mất 10,37 giây, các epoch sau gần bằng bản tự viết. Bảng này đo cả vòng train, nên chưa đủ để so tốc độ riêng của phép MSA.



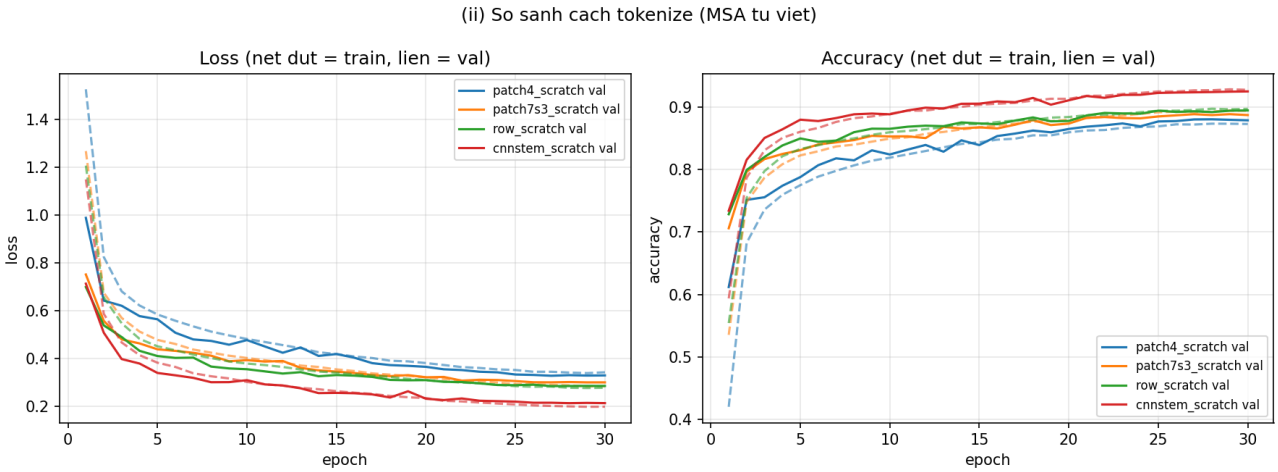
Hình 3.5. Đường cong của hai bản MSA với patch4.

(ii) So sánh tokenizer với cùng MSA tự viết

CNN-stem đạt test cao nhất, 92,25%, hơn patch4 3,75 điểm với thêm 17.920 tham số (12,89%). Nhóm cho rằng các lớp convolution giúp mô hình học mẫu cục bộ trước attention, nhưng chưa thử một bản cân bằng số tham số để tách riêng tác động này.

Patch chồng lấn đạt 89,54%, hơn patch4 1,04 điểm. Row đạt 89,14% với chỉ 28 token ảnh, cũng cao hơn patch4 với 49 token. Như vậy, trong thí nghiệm này, nhiều token hơn không đi kèm accuracy cao hơn. Row có validation cao hơn patch chồng lấn nhưng test thấp hơn, nên thứ tự của hai cách này chưa ổn định.

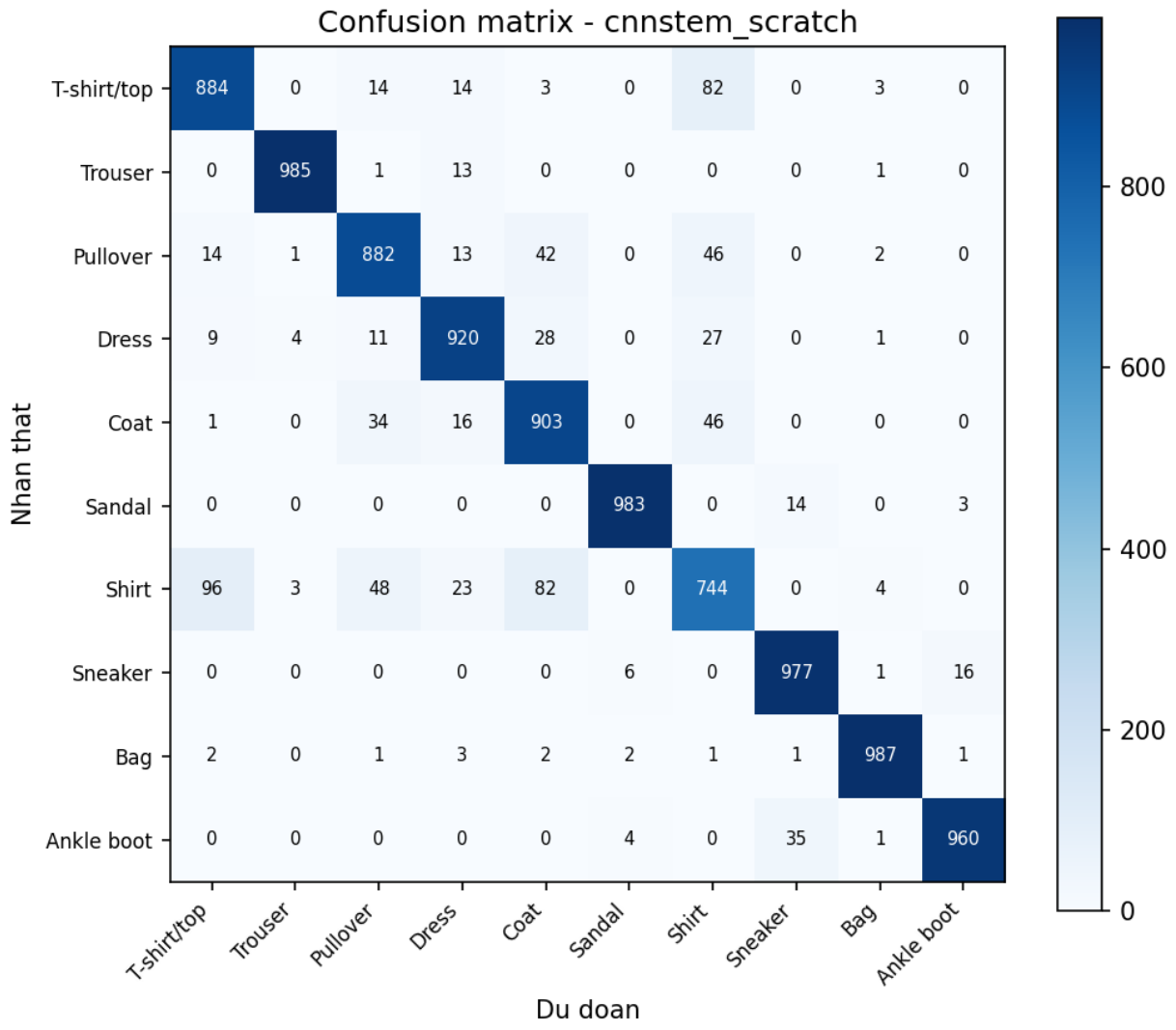
Về thời gian, S^2 của patch7s3 gấp khoảng 5 lần của row (4.225 so với 841), nhưng thời gian mỗi epoch chỉ chênh khoảng 3% (8,82 và 8,53 giây). Attention có vẻ chưa phải phần tốn thời gian nhất ở kích thước này. Nạp dữ liệu có thể là nút thắt, nhưng nhóm chưa đo riêng.



Hình 3.6. Đường cong của bốn tokenizer với MSA tự viết.

Ở epoch cuối, train accuracy và validation accuracy của cả năm mô hình chênh dưới 1 điểm, ví dụ CNN-stem 92,72% và 92,46%. Epoch tốt nhất đều từ 27 trở đi. Các mô hình chưa có dấu hiệu overfit rõ, nên có thể thử mô hình lớn hơn hoặc train lâu hơn.

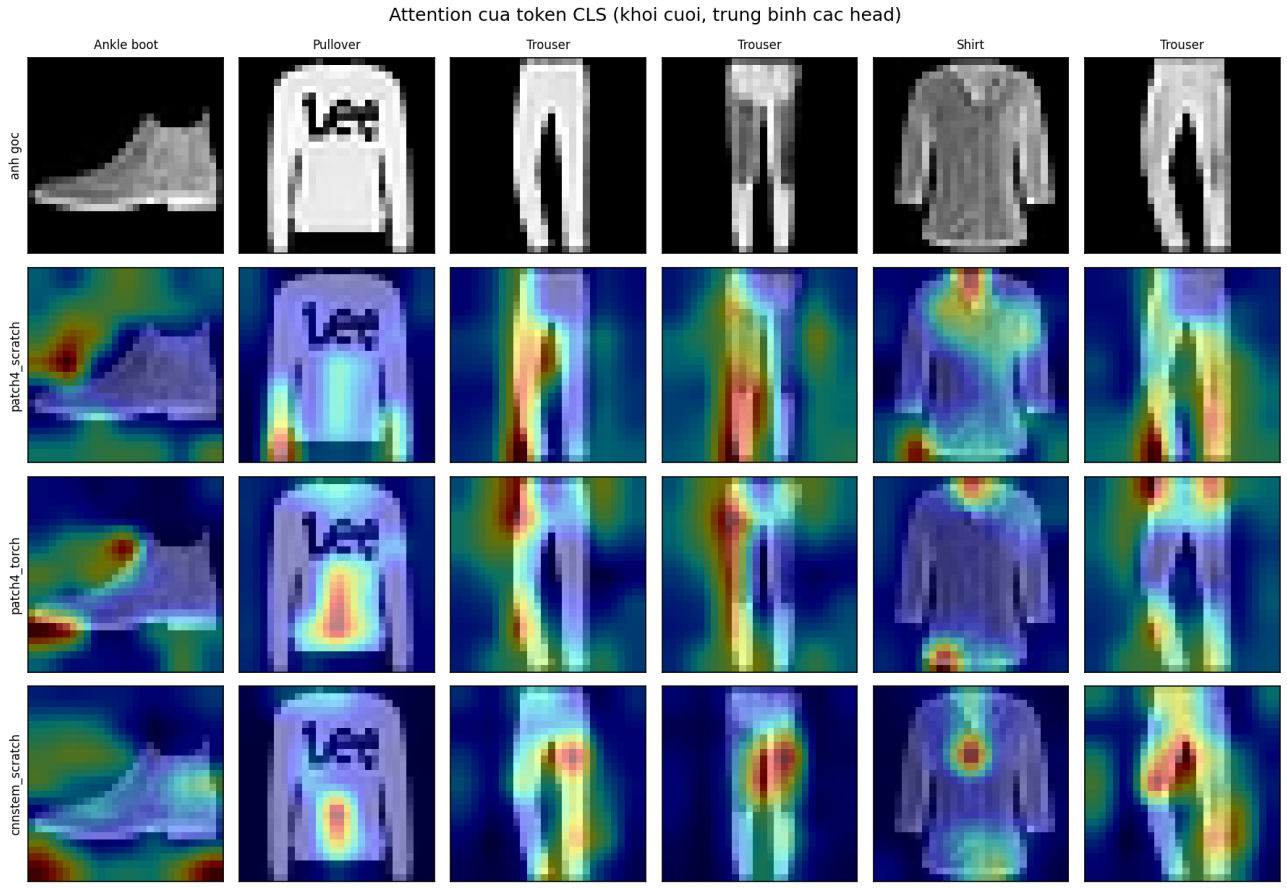
3.7. Phân tích lỗi và hình attention



Hình 3.7. Confusion matrix của cnnstem_scratch, mô hình có validation accuracy cao nhất.

Hàng là nhãn thật, cột là dự đoán. Mô hình đúng 9.225 và sai 775 ảnh trên 10.000. Các ô nhầm lớn nhất là Shirt → T-shirt/top (96 ảnh), T-shirt/top → Shirt (82 ảnh) và Shirt → Coat (82 ảnh).

Shirt có recall thấp nhất, 74,40%, trong khi Bag đạt 98,70%. Ngoài Shirt, lỗi còn tập trung giữa Pullover và Coat. Giày cũng nhầm nhau nhưng ít hơn, lớn nhất là Ankle boot → Sneaker (35 ảnh). Kết quả này khớp với nhận xét ở mục 1.2 rằng các lớp áo có đường viền giống nhau, dù nhóm chưa xem riêng từng ảnh lỗi.



Hình 3.8. Trọng số từ CLS đến token ảnh ở khối cuối, trung bình các head.

Hình này lấy hàng query CLS, bỏ cột CLS, trung bình bốn head rồi nội suy lên 28×28. Vùng nóng khác nhau khá nhiều giữa patch4_scratch và patch4_torch trên cùng một ảnh, dù hai mô hình có accuracy gần bằng nhau. Với patch4, một số vùng nóng nằm ngoài vật thể, như ở ảnh Ankle boot. Với CNN-stem, vùng nóng ở các ảnh Trouser và Shirt tập trung ở phần giữa vật thể hơn. Hình chỉ cho biết CLS đặt trọng số ở đâu, chưa cho biết vùng đó quyết định dự đoán.

3.8. Kết luận E2

Nhóm đã cài đặt MSA tự viết, MSA của PyTorch và bốn cách lấy token. MSA tự viết khớp với bản PyTorch trong hai ca kiểm tra, và accuracy của hai bản gần như nhau. CNN-stem cho kết quả cao nhất cả trên validation lẫn test; row và patch chồng lấn nhỉnh hơn patch4. Lỗi chủ yếu nằm ở các lớp áo, nhất là Shirt.

Kết quả dựa trên một seed và một lần chạy cho mỗi cấu hình, các tokenizer chưa được cân bằng số tham số, và thời gian chưa được đo riêng cho MSA. Bước tiếp theo là lặp lại với nhiều seed và đo riêng thời gian nạp dữ liệu.

4. So sánh E1 và E2

CNN của E1 và CNN-stem của E2 có cùng hai khối convolution (19.008 tham số). Điểm khác là phần phía sau: E1 trải feature map rồi đưa qua các lớp Linear, còn E2 coi mỗi vị trí trên feature map là một token và đưa qua bốn khối MSA. Vì vậy, CNN-stem có thể xem như một cách thay phần phân loại của CNN bằng MSA.

Chỉ số	CNN (E1)	CNN-stem (E2)
Tổng tham số	421.834	156.938
Tham số sau phần convolution	402.826	137.930
Test accuracy (%)	91,45	92,25
Macro-F1 (%)	91,39	92,22
Số ảnh test sai	855	775
Recall của Shirt (%)	69,60	74,40
Shirt → T-shirt/top (ảnh)	117	96
Shirt → Coat (ảnh)	97	82

CNN-stem đúng thêm 80 ảnh test với số tham số bằng khoảng 37% của CNN. Phần lớn cải thiện về Shirt đến từ việc giảm nhầm sang T-shirt/top và Coat, trong khi T-shirt/top → Shirt gần như không đổi (81 và 82 ảnh). Attention cho phép mô hình tổng hợp thông tin giữa các vị trí xa nhau trên ảnh, nên nhóm cho rằng đây có thể là một lý do của cải thiện này.

Kết quả này chưa đủ để quy toàn bộ cải thiện cho MSA, vì E2 còn dùng AdamW, warmup và cosine, train 30 epoch so với 15 epoch của CNN, và mỗi mô hình mới chạy một seed. Để kiểm tra riêng tác động của MSA, cần train hai mô hình với cùng optimizer, lịch learning rate và số epoch.

Nếu bỏ convolution, kết quả thấp hơn: ba cách lấy token còn lại đạt 88,50% đến 89,54%, cao hơn MLP (87,39%) nhưng thấp hơn CNN (91,45%). Với tập dữ liệu và kích thước mô hình này, MSA cho kết quả tốt nhất khi đi sau các lớp convolution, chứ chưa thay thế được convolution.

Phụ lục

P.1. Nguồn số liệu

Nội dung	File trong repo
Split dùng chung	E1/splits/train_idx.npy, val_idx.npy, split_meta.json
Môi trường và output khi chạy	E1/notebooks/E1_Colab.ipynb, E2/notebooks/E2_Colab.ipynb
Kết quả E1	E1/results/comparison_table.json; E1/results/<mô_hình>/history.csv, summary.json
Cấu hình E2	E2/src/experiments.py
Đối chiếu MSA	E2/results/msa_check.json
Kết quả E2	E2/results/comparison_table.json; E2/results/<tên_thí_nghiệm>/history.csv, summary.json

P.2. Khai báo hỗ trợ AI

Nhóm dùng công cụ AI (Codex, Claude Code) như sau. AI tạo sườn code và ví dụ, đồng thời gợi ý các thí nghiệm cần làm. Nhóm sửa đổi và bổ sung thêm template rồi chạy thí nghiệm. Sau khi có kết quả, AI tổng hợp và viết báo cáo, nhóm sửa lại sau đó. AI cũng hỗ trợ viết test, vẽ sơ đồ, dàn trang PDF và chỉnh README.

Tài liệu tham khảo

[1] Han Xiao, Kashif Rasul, Roland Vollgraf. Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms (2017). Kho dữ liệu chính thức.

[2] Ashish Vaswani và cộng sự. Attention Is All You Need. NeurIPS 2017. arXiv:1706.03762.

[3] Alexey Dosovitskiy và cộng sự. An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. ICLR 2021. arXiv:2010.11929.

[4] PyTorch. MultiheadAttention, tài liệu chính thức.